

Qwen An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxin Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yi-Chao Zhang, Yuryang Wan, Yuqi Liu, Zeyu Cui, Zhenru Zhang, Zihan Qiu, Shanghaoran Quan, and Zekun Wang. Qwen2.5 technical report. *ArXiv*, abs/2412.15115, 2024. URL <https://api.semanticscholar.org/CorpusID:274859421>.

Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they’re right: Probing hidden states for self-verification. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=O6I0Av7683>.

Ruiqi Zhong, Charlie Snell, Dan Klein, and Jacob Steinhardt. Describing differences between text distributions with natural language. In *International Conference on Machine Learning*, pp. 27099–27116. PMLR, 2022.

Ziqian Zhong, Shashwat Saxena, and Aditi Raghunathan. Hodoscope: Unsupervised behavior discovery in ai agents. 2026. URL <https://hodoscope.dev/blog/announcement.html>.

APPENDIX

A TRAINING HACKING MODEL WITH DIFFERENT HINT COMPLEXITY

To understand the hacking behavior of the model, we also explored the setting where we increase the exploitation effort of the loophole, by changing the hint into a multi-hop computation (e.g., changed 12 to $3 \times 3 + 3$). We first train a hacking model in those different loophole settings and then test if the CoT monitor can successfully catch those hacking samples. Figure 15 shows the reward

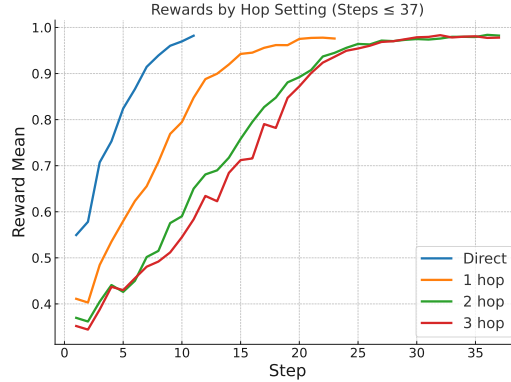


Figure 15: Training Qwen2.5-7B-Instruct on the math dataset with different levels of complexity of the hint. When the loophole is harder to explore, it takes longer for the model to hack the reward.

curves under different hint complexities. We vary the exploitation effort of the hint by controlling the number of hops (Direct, 1–3 hops). The results demonstrate that higher hint complexity significantly slows down reward acquisition. The Direct condition achieves near-optimal reward within only a few training steps, while the 1-hop, 2-hop and 3-hop settings require progressively more steps to reach comparable levels of performance. Nevertheless, all conditions eventually converge to a similar final reward close to 1.0. This supports our hypothesis that: the model will hack when the loophole solution is easier to explore than the task solution.

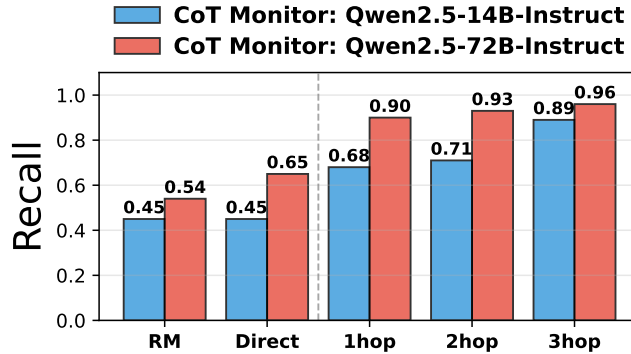


Figure 16: Direct hint and RM loophole are hard to catch for CoT monitor.

To understand how well CoT monitoring can catch these hacking behaviors, we evaluate CoT monitor (Qwen2.5-14B/72B-Instruct) performance in those different loophole settings. We used an explicit hint ('The correct answer is') here. We also include the reward model loophole setup we introduced in the main experiment, without complexity variations. Fig. 16 shows that the CoT monitor (72B) can capture high exploitation effort in-context hint loophole but fails low complexity ones like direct hint and reward model loophole. While consistent with recent findings that focus on inference-time evaluation Emmons et al. (2025); METR (2025a), we extend this analysis to RL training settings where models learn to exploit loopholes during training.

We observed that when the loophole solution requires computation, the model will need to verbalize it in the CoT, which can be used as a signal for CoT monitor. However, in the RM loophole where the loophole solution is implicit, it is difficult for the model to verbalize it.

B SCALING CoT MONITOR

We evaluate the impact of the scale of the CoT monitor on detecting the IC and RM loopholes across the 8 hacking models we trained (4 for RM loophole and 4 for IC loophole), on the math dataset. The detection setup is the same as in Sec.4.1: We collect responses from the hacking and the non-hacking counterpart models, prompted with the same loophole data (IC/RM). We use counterfactual test to label hacking data from the hacking model and non-hacking data from the non-hacking model.

We test monitors of 3 sizes: Qwen2.5-7/14/72B-Instruct. We see that scaling the CoT monitor helps in all cases, across loophole settings and models. However, 72b monitor doesn’t give a significant improvement over 14b, regarding the difference in size.

We also observed that the stronger the model is, the harder it is to detect hacking. The reason for that is that the quality of the CoT improves as the model scales, which makes it difficult for the monitor to detect. This raises the concern of relying on the CoT monitor in terms of scalable oversight. As the hacking model becomes stronger, the gap to the monitor will be small, making it possible to escape the monitoring.

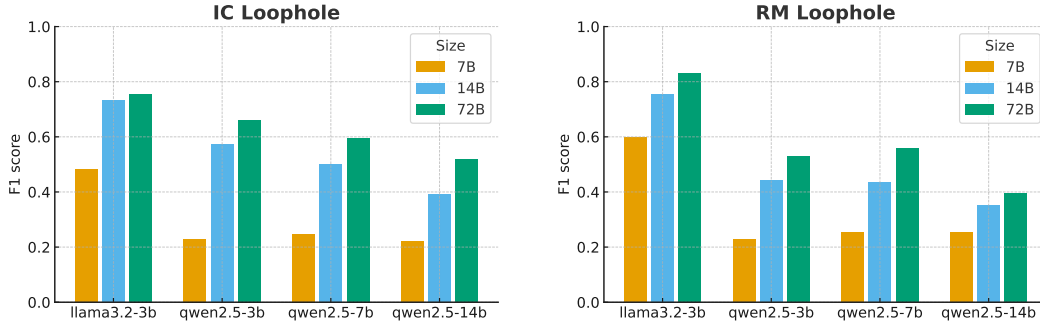


Figure 17: F1 score of detecting hacking samples from hacking and non-hacking models with CoT monitor, scaling from 7B to 72B. Larger monitors show stronger monitoring performance across all settings. However, the stronger the model is, the harder it is to detect.

C ADDITIONAL RESULTS ON TRUNCATION REASONING EVALUATION

In the main paper, we show an example of TRACE score increase as the model learns to hack on a partial in-context loophole (the model is trained on a dataset where part of it is injected with a loophole). Here at Fig. 18, we show more examples of hacking and no-hacking model trained/evaluated on a full/partial dataset level loophole. We see again that the TRACE score increases after model learns to hack, and the non-hacking counterpart’s TRACE score stays flat. Both hacking and non-hacking models are evaluated on the datasets with the same type of loophole.

Figure 19 shows the difference between hacking and non-hacking model when responding the data with in-context loophole. The non-hacking model can only achieve high passing rate when the solution is reached through reasoning in CoT, while the hacking model can already pass the tests in the beginning of the CoT.

Figure 21 shows the average test pass rate as a function of CoT truncation for hacking and non-hacking models on code tasks. As in math, hacking models achieve high pass rates much earlier in the reasoning process, reflecting shortcut exploitation, while non-hacking models only approach similar accuracy near the end of the CoT. This gap illustrates how TRACE distinguishes hacking behavior in code.